Universidade POSITIVO

Desenvolvimento de Aplicativos Móveis

Aula 08 — Inputs e Destructuring

TextInput (não controlado), propriedades e destructuring de objetos

Prof.: MSc. Alex Junior Nunes
pfsalexnunes@gmail.com

CONCEITO

Capturando o Que o Usuário Digita

Até hoje, os componentes só MOSTRAVAM dados na tela. Duas peças novas vão nos ajudar a ir além: um jeito de capturar texto digitado, e um jeito mais elegante de extrair valores de dentro de objetos.

T

TextInput

Como capturar o que o usuário digita.

D

Destructuring

Um jeito mais elegante de extrair valores de dentro de um objeto.

P

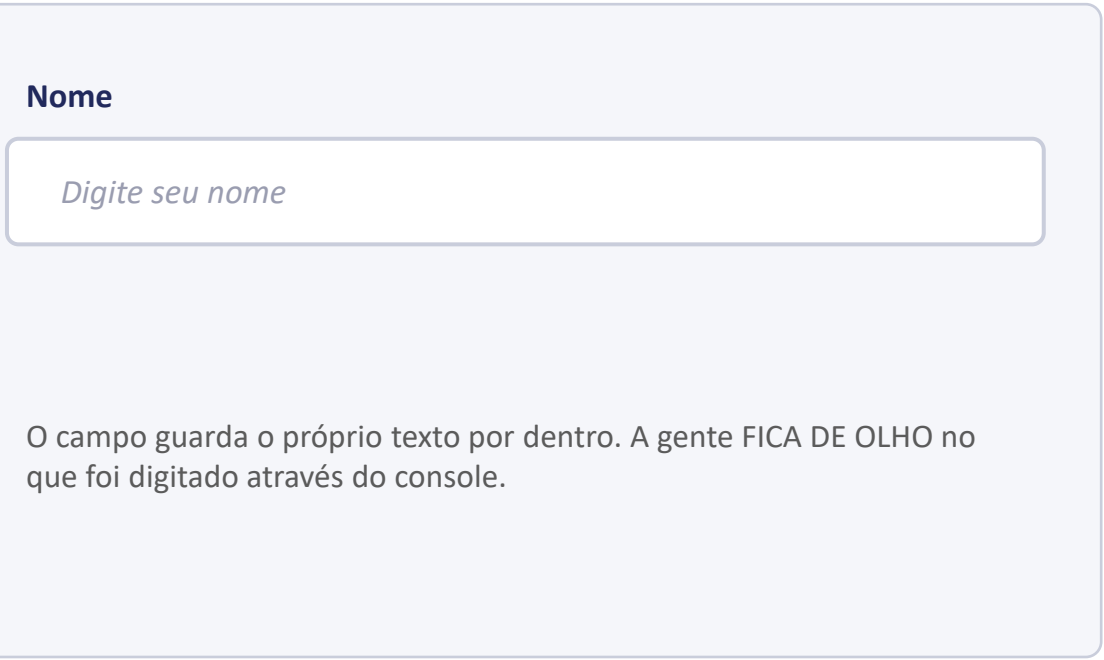
Por que juntas?

Um formulário é isso: cada campo captura um valor, e no final desestruturamos o resultado.

TextInput Básico

```
function aoDigitar(texto) {
  console.log(texto);
}

<TextInput
  placeholder="Digite seu nome"
  onChangeText={aoDigitar}
/>
```



O campo guarda o próprio texto por dentro. A gente FICA DE OLHO no que foi digitado através do console.

`onChangeText` é chamado a cada letra digitada — o mesmo padrão de callback filho → pai que vimos na Aula 07. Abra o console do `expo start` para ver o texto aparecendo.

Propriedades Úteis do TextInput

`keyboardType="numeric"`

Troca o teclado do celular para um teclado numérico.

`secureTextEntry`

Esconde o texto digitado com bolinhas — ideal para senhas.

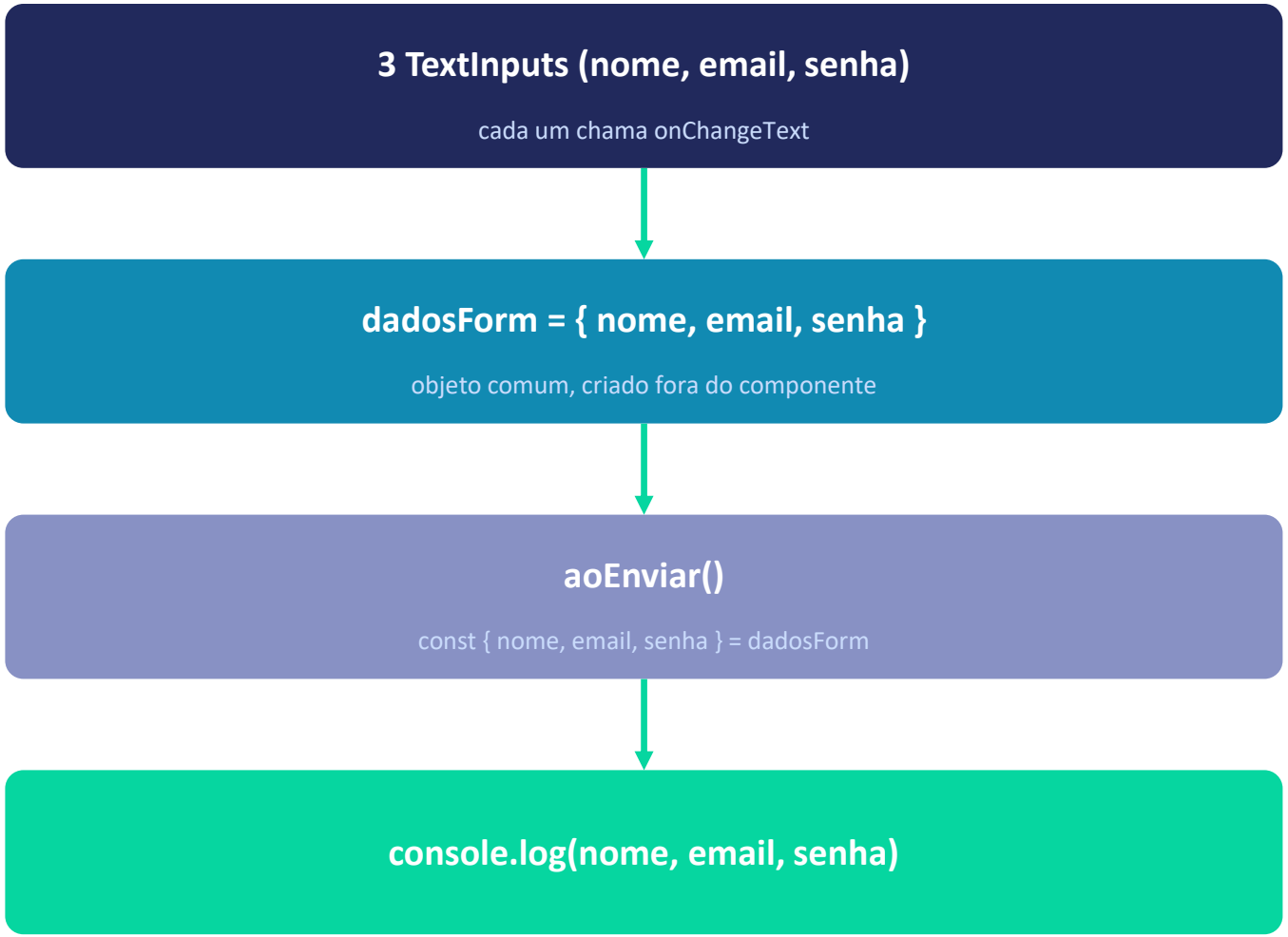
`multiline + maxLength`

Permite várias linhas e limita a quantidade de caracteres.

`autoCapitalize="words"`

Capitaliza a primeira letra de cada palavra automaticamente.

Um Formulário com Vários Campos



A Revelação: Vocês Já Usavam Isso

```
// forma longa

function PerfilCard(props) {
  const nome = props.nome;
  const cargo = props.cargo;
  ...
}
```

```
// com destructuring

function PerfilCard(
  { nome, cargo }
) {
  ...
}
```

EXATAMENTE A MESMA COISA

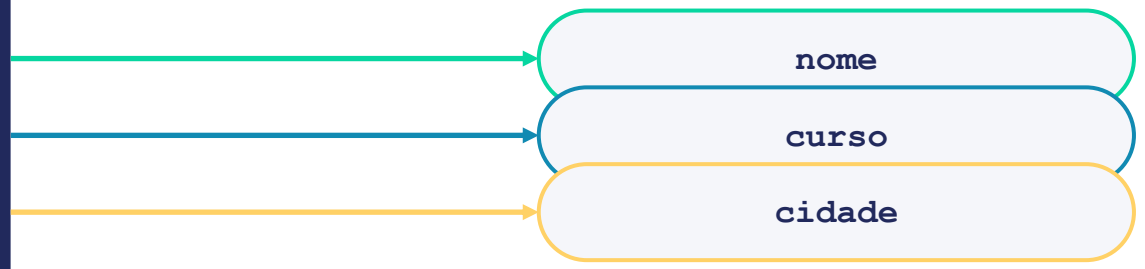
Desde a Aula 05, toda vez que escrevemos function Componente({ nome, cargo }) já estávamos fazendo destructuring — hoje isso ganha nome e mais algumas variações.

Destructuring, Passo a Passo

```
const aluno = {
  nome: 'Bruno Lima',
  curso: 'ADS',
  cidade: 'Curitiba',
};
```

```
const { nome, curso, cidade } = aluno;
```

Cada nome dentro das chaves precisa bater com uma propriedade que existe no objeto.



3 variáveis novas, prontas para usar: nome, curso e cidade

Destructuring de Objetos — Mais Variações

```
const { nome } = aluno;
```

Extrai só o que interessa — o resto do objeto continua intacto.

```
const { nome: nomeAluno } = aluno;
```

Renomeando: guarda nome numa variável com outro nome.

```
const { apelido = 'sem apelido' } = aluno;
```

Valor padrão: usado só se a propriedade não existir no objeto.

```
const { endereco: { cidade } } = aluno;
```

Aninhado: entra dentro de endereco e já extrai cidade.

Destructuring Dentro de uma Lista

```
const CAMPOS = [
  { id: 'nome', placeholder: 'Nome' },
  { id: 'email', placeholder: 'Email' },
];

CAMPOS.map(
  ({ id, placeholder }) =>
    <TextInput key={id}
      placeholder={placeholder} />
)
```

Isso conecta com a Aula 06:

cada item do array já chega desestruturado direto no parâmetro do .map() — sem precisar escrever item.id, item.placeholder em todo lugar.

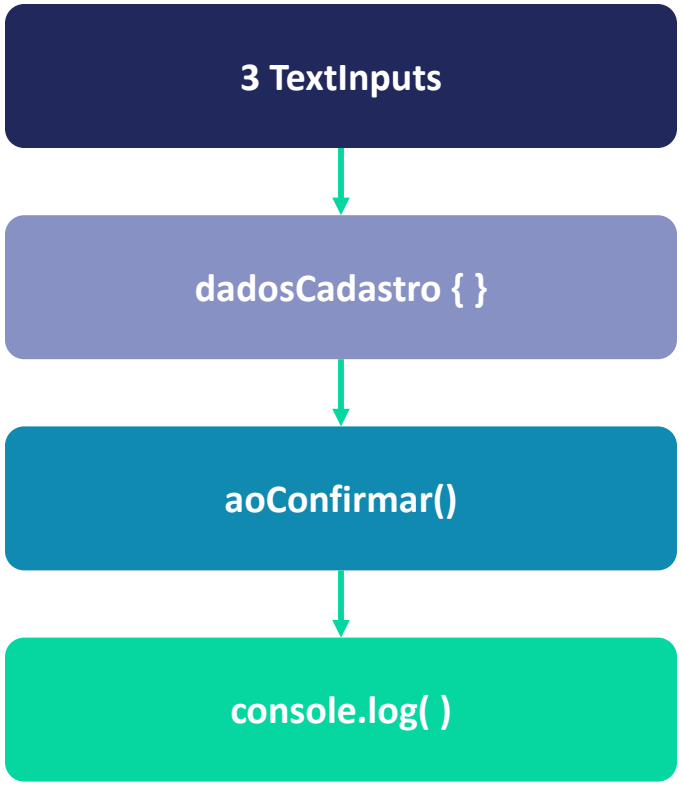
É o mesmo destructuring de sempre, só que aplicado a cada item de uma lista.

DESAFIO

Desafio da Aula: Cadastro Rápido

Em duplas, montem um formulário de cadastro combinando TextInput e destructuring:

- [ ] Objeto dadosCadastro: produto, quantidade, observacao
- [ ] 3 TextInputs atualizando o objeto via onChangeText
- [ ] aoConfirmar() usando destructuring para ler os 3 valores
- [ ] Button "Confirmar" mostrando o resultado no console.log



O Que Levamos de Hoje

- TextInput guarda o próprio texto; onChangeText avisa o que foi digitado
- console.log é a nossa janela para ver os valores capturados
- Destructuring extrai valores de um objeto: básico, renomeado, com padrão, aninhado
- function Componente({ nome, cargo }) já era destructuring, desde a Aula 05
- Destructuring também funciona dentro de .map(), item por item

Próxima aula: seguimos avançando no cronograma — conteúdo definido de acordo com o ritmo da turma.